

ASK PHASE

Executive Summary

Component	Overview
Situation	PHARx (London pharma distributor) faced severe financial write-offs at its Park Royal depot due to two competing issues: expensive specialty drugs expiring on shelves, and common daily medications running completely out of stock.
Task	Lead a data-driven operational audit of 12 months of transactional sales and inventory logs commissioned by the Head of Logistics and Corporate Finance Team .
Action	Combine and fix messy company files in Google BigQuery , set up automated checks to catch data errors, and analyze how quickly different products are being sold.
Result	Provide clear ordering guidelines to stop common medications from going out of stock (protecting steady sales) and adjust purchasing to match actual buyer demand (preventing costly expiration waste).

Project Scope & Core Business Objectives

To address both inventory waste and stockout risks effectively, the analysis isolates two distinct operational tracks using a 12-month transactional dataset:

1. Minimizing Specialty Inventory Waste (Overstocking Risk)

The Problem: High-cost, low-turnover specialty medications sit in stock past their shelf life, causing them to expire and go to waste before being distributed, resulting in severe financial losses.

The Strategy: Analyze **Days Sales of Inventory (DSI)** and inventory turnover rates against strict product expiration windows to adjust procurement volumes to match actual, low-volume demand patterns.

2. Eliminating Stockouts on High-Demand Daily Medications (Understocking Risk)

The Problem: Sudden spikes in demand for critical, everyday maintenance drugs (e.g., Diabetology and Cardiology) lead to stockouts, resulting in lost revenue and risking partner pharmacies turning to competing distributors.

The Strategy: Analyze daily order frequencies and **Supplier Lead Times** to calculate exact, optimized safety stock thresholds to guarantee continuous product availability.

Project Hypotheses

Hypothesis 1 (Inventory Waste): I expect the data will show that high-cost specialty medications account for the majority of financial losses. Because these products are expensive, purchasing them in **large, infrequent bulk quantities**—combined with low customer demand and rigid expiration windows—inadvertently leads to severe financial waste as excess stock reaches its expiration date before it can be distributed.

Hypothesis 2 (Out-of-Stock Risk): I expect the data will show that common everyday prescriptions or medication go out of stock because of supplier delivery delays or rigid ordering schedules that fail to adjust to sudden spikes in sales.

PREPARE PHASE

Data Storage & Structure

Data Location: All datasets are securely stored in a centralized Cloud Storage bucket and directly mapped into Google BigQuery as a dedicated relational schema.

Data Organization: The data is structured in three core files:

TABLE 1 'dim_products' → *Product IDs, names, categories, unit costs, and shelf-life metrics.*

TABLE 2 'fact_sales' → *12 months of daily transactional sales records, pharmacy customer IDs, and quantities sold.*

TABLE 3 'fact_inventory_logs' → *Daily warehouse stock tracking, showing exactly when items were received, stored, or flagged as out of stock.*



Credibility, Privacy & Compliance (ROCCC Assessment)

Bias & Credibility: Real-world pharmaceutical supply chain logs are heavily restricted due to privacy laws and corporate confidentiality. To maintain absolute data integrity without compromising legal standards, the data was custom-engineered to match the exact mathematical distributions and relational logic of a real distributor. It fully satisfies the ROCCC framework by being Reliable, Comprehensive, and Current (2025) over a fixed 12-month timeline.

Privacy, Security & Licensing: Because the dataset contains entirely synthetic business operations, it presents zero data privacy risks (no real patient, pharmacy, or corporate financial records are exposed). Standard database access controls are enforced within Google BigQuery to mimic secure corporate data governance.

Integrity Checks & Data Challenges

Answering the Business Task: By connecting sales trends (fact_sales) directly to warehouse stock levels (fact_inventory_logs) and shelf-life constraints (dim_products), the data provides a complete, unbroken view of exactly when and why products either expire or go out of stock.

Identified Data Challenges: To simulate a realistic data cleaning scenario, the raw files intentionally contain common system anomalies that must be resolved before analysis:

- Mismatched and irregular date formats across the different logs.
- Inconsistent string formatting in product category names.
- Missing (null) values in inventory records that represent zero-stock days.

PROCESS PHASE

1. Tool Selection & Justification

Tool Chosen: Google BigQuery (SQL)

Why: The `pharx_data` dataset contains multi-table relational data (dimension and fact tables) requiring structural data validation, primary key transformations, and conditional parsing loops. BigQuery provides a robust, scalable cloud infrastructure to execute advanced date-parsing functions (`SAFE.PARSE_DATE`) and conditional aggregation checks (`COALESCE`) efficiently without local processing limitations.

2. Data Integrity Strategy

Data integrity was locked in across three distinct dimensions before any final analytics tables were built:

- **Uniqueness:** Audited all primary keys (`product_id`, `log_id`, `transaction_id`) to verify transactional limits. Grouping metrics were validated using `HAVING COUNT(*) > 1` checks to prove zero duplicate entries exist.
- **Completeness:** Full column sweeps were executed across all fields to look for `NULL` values or empty text strings (`''`). A notable gap of 93 missing `expiry_date` fields was discovered in the inventory logs, which was isolated and flagged for upstream data engineering review.
- **Format Consistency:** Text identifiers were unified to eliminate variations in spaces or casing using `TRIM(UPPER())`.

3. Advanced Cleaning Steps & Verification

To ensure the data is safe, clean, and ready for analysis, a multi-tiered conditional fallback pipeline was implemented to parse dates from string formats to true database `DATE` objects. By nesting individual format rules inside a `COALESCE` clause paired with `SAFE.` prefixes, mixed delimiter formats (hyphens vs. slashes) were cleaned without query errors or data loss.

Final status verification checks confirmed that all processing issues were mitigated, leaving structural date field anomalies at exactly 0 records.

Table 1:	dim_product
Columns:	product_id, product_name, drug_category, unit_cost, unit_price

Approach: Uniqueness: Conducted isolated, vertical column audits rather than full-row checks to identify duplicates.

Completeness: Swept identifying text fields for missing values (NULL), while conducting a manual visual audit for the numeric fields.

Attention to Detail: * Targeted product_id (Primary Key) to ensure no two different drugs were assigned the same identifier, and verified that no rows were left blank or unassigned.

- Targeted product_name to check if a single drug asset was accidentally duplicated under different IDs.
- Applied TRIM(UPPER()) to both text fields to neutralize hidden spaces or inconsistent capitalizations.
- Intentionally skipped drug_category and pricing columns, as these have repeating values, which are structurally correct.
- **Leveraged Scale for Efficiency:** Recognized that because this dimension table is highly compact (only 50 rows), a **manual visual scan of the FLOAT columns** (unit_cost and unit_price) was the most efficient way to confirm for NULL values.

Check for duplicate Product ID's

#Query A: Check for duplicate Product Id's

```
SELECT
  TRIM(UPPER(product_id))
FROM `skilful-answer-498915-k5.pharx_data.dim_products`
GROUP BY TRIM(UPPER(product_id))
HAVING COUNT(*) > 1;
```

#Query B: Check for duplicate Product Names

```
SELECT
  TRIM(UPPER(product_name))
FROM `skilful-answer-498915-k5.pharx_data.dim_products`
GROUP BY TRIM(UPPER(product_name))
HAVING COUNT(*) > 1
```

#Result: 0 product_name's or Id's found

Check for NULL values

#Check for NULLs or Blank Values

```
SELECT
    product_id,
    product_name
FROM `skilful-answer-498915-k5.pharx_data.dim_products`
WHERE product_id IS NULL
    OR TRIM(product_id) = ''
    OR product_name IS NULL
    OR TRIM(product_name) = ''
```

#No NULLs/ BLANKS detected

Final Data Transformation & Standardized View:

This query standardizes text formatting, converts financial columns to exact decimal scales, and creates a clean production table (**cleaned_dim_products**) ready for the analysis phase.

```
CREATE OR REPLACE TABLE `skilful-answer-498915-k5.pharx_data.cleaned_dim_products` as
SELECT
    TRIM(UPPER(product_id)) AS product_id, #Trim hidden spaces and capitalize product IDs
    TRIM(INITCAP(product_name)) AS product_name, #Trim spaces and apply Title Case to product names
    TRIM(INITCAP(drug_category)) AS drug_category, #and category names
    unit_cost,
    unit_price
FROM `skilful-answer-498915-k5.pharx_data.dim_products`
    ORDER BY unit_cost DESC; #Sort results by highest unit cost first
```

Table 2:	fact_inventory_logs
Columns:	log_id, log_date, product_id, current_stock_on_hand, expiry_date, supplier_lead_time_days

Approach: Verified that every transaction has a unique ID number to ensure no sales or inventory movements were counted twice.

Completeness: Evaluated the entire dataset for missing values (NULL or empty strings), identifying NULL values in the expiration date records while verifying that all other critical fields are 100% complete.

Attention to Detail: * Recognized that dates, quantities, and product IDs *can* repeat across standard inventory movements, thus only focusing on the log_id, which must be strictly unique to prevent any transactional double-counting.

- Utilized CAST(log_id AS STRING) to standardize the data type format, proactively preparing the key for seamless future table joins.
- Intentionally skipped text-cleaning functions (TRIM/UPPER), identifying that system-generated numeric IDs cast to strings are inherently free of spacing or casing defects.
- **Data Quality Insight:** Uncovered **93 rows** with missing expiry_date values across critical medical categories (such as Cardiology, Antibiotics, and Diabetology). Since these medications require expiry dates by law, this was flagged as a data integrity issue from the supplier or data pipeline. Kept the rows intact to preserve accurate stock balances, but added a recommendation to escalate this to the data engineering team.
- Identified inconsistent text patterns in the date fields (log_date and expiry_date), which featured a mixture of delimiters (hyphens vs. slashes) and ordering structures (day-first vs. year-first). Implemented a multi-layered parsing strategy to protect against query crashes and data erasure.
- Uncovered an underlying format trap where standard parsing would have silently converted 715 valid hyphenated dates (YYYY-MM-DD) into NULL values. This formatting anomaly was completely resolved by implementing a fallback pipeline.

Identify duplicate log_id's

```
SELECT
  CAST(log_id as STRING)          #1. Transform primary key from integer to text for future joins
FROM `skilful-answer-498915-k5.pharx_data.fact_inventory_logs`
GROUP BY CAST(log_id as STRING)  #2. Group by log_id
HAVING COUNT(*) > 1;             #3. Filter results to display only IDs appearing more than once

# Result: 0 duplicate log_id's found.
```

Check for NULL values

```
SELECT
log_id,
log_date,
product_id,
current_stock_on_hand,
expiry_date,
supplier_lead_time_days
FROM `skilful-answer-498915-k5.pharx_data.fact_inventory_logs`
WHERE log_id IS NULL
OR log_date IS NULL
OR TRIM(log_date) = ''
OR product_id IS NULL
OR current_stock_on_hand IS NULL
OR expiry_date IS NULL
OR TRIM(expiry_date) = ''
OR supplier_lead_time_days is NULL

#93 NULL results under expiry_date. Flag to the data engineering or supplier team to fix the data pipeline
```


Final Transformation & Table Creation

Logic Breakdown: To resolve the mixed string date formats without crashing the pipeline or generating **NULL** values, the extraction relies on **COALESCE** paired with **SAFE.PARSE_DATE**.

- **SAFE.PARSE_DATE**: Functions as a safe interpreter. It attempts to deconstruct and translate the raw text string using a designated pattern. If the structural layout does not match (e.g., trying to read hyphens with a slash template), it returns a database **NULL** instead of a catastrophic query error.
- **COALESCE**: Acts as an ordered evaluation bubble. It tests each parsing calculation sequentially from left to right, locking in the first successful conversion and skipping subsequent rules once a valid date is resolved.

```
CREATE OR REPLACE TABLE
`skilful-answer-498915-k5.pharx_data.cleaned_fact_inventory_logs` AS
SELECT
CAST(log_id as STRING) as log_id,
#Multi-format fallback bubble ensures all 715 variant rows are preserved
COALESCE(
    SAFE.PARSE_DATE('%d-%m-%Y', log_date),
    SAFE.PARSE_DATE('%Y-%m-%d', log_date),
    SAFE.PARSE_DATE('%d/%m/%Y', log_date),
    SAFE.PARSE_DATE('%Y/%m/%d', log_date)
) as log_date,
TRIM(UPPER(product_id)) as product_id,
current_stock_on_hand as in_stock,
#Replicates identical logic for target expiration timelines
COALESCE(
    SAFE.PARSE_DATE('%d-%m-%Y', expiry_date),
    SAFE.PARSE_DATE('%Y-%m-%d', expiry_date),
    SAFE.PARSE_DATE('%d/%m/%Y', expiry_date),
    SAFE.PARSE_DATE('%Y/%m/%d', expiry_date)
) as expiry_date,
supplier_lead_time_days
FROM `skilful-answer-498915-k5.pharx_data.fact_inventory_logs`
    ORDER BY log_date DESC;

#Final Validation: Confirmed that post-execution NULL checks on log_date dropped down
to exactly 0 rows.
```

Table 3:	fact_sales
Columns:	transaction_id, sales_date, product_id, quantity_sold, total_revenue

Approach: Validated the transactional unique constraints of the sales system by auditing the point-of-sale identifiers.

Completeness: Evaluated the complete transaction history for missing or empty fields, confirming that all primary data fields are 100% structurally complete with no missing values.

Attention to Detail:

- Duplicates: Isolated the `transaction_id` as the unique tracking number for the sales record, making sure every individual sale is only counted once.
- Applied `TRIM(UPPER())` to guard against potential formatting, casing, or spacing variations introduced during system text generation.
- Bypassed secondary columns (like date, store, or customer IDs) where recurring data entries represent normal retail transaction histories.
- Kept the original `FLOAT64` data type for `total_revenue` as-is to preserve raw decimal precision for future calculations.
- Used `COALESCE` with `SAFE.PARSE_DATE` to handle the mixed date string formats and prevent query errors.

Check for duplicate Sales Transaction Id's

```
SELECT
TRIM(UPPER(transaction_id))
FROM `skilful-answer-498915-k5.pharx_data.fact_sales`
GROUP BY TRIM(UPPER(transaction_id))
HAVING COUNT(*) > 1
```

Result: 0 duplicate transaction_id's found.

Check for NULL's

```
SELECT*
FROM `skilful-answer-498915-k5.pharx_data.fact_sales`
WHERE transaction_id IS NULL
OR TRIM(transaction_id) = ''
OR sales_date IS NULL
OR TRIM(sales_date) = ''
OR product_id IS NULL
OR TRIM(product_id) = ''
OR quantity_sold IS NULL
OR total_revenue IS NULL;

#No NULLS/ BLANKS detected
```

Final Transformation & Table Creation

```
CREATE OR REPLACE TABLE `skilful-answer-498915-k5.pharx_data.cleaned_fact_sales` AS
SELECT
TRIM(UPPER(transaction_id)) as transaction_id,
COALESCE(
SAFE.PARSE_DATE('%d/%m/%Y', sales_date),
SAFE.PARSE_DATE('%Y/%m/%d', sales_date),
SAFE.PARSE_DATE('%d-%m-%Y', sales_date),
SAFE.PARSE_DATE('%Y-%m-%d', sales_date)
) as sales_date,
TRIM(UPPER(product_id)) as product_id,
quantity_sold,
total_revenue
FROM `skilful-answer-498915-k5.pharx_data.fact_sales`
ORDER BY total_revenue DESC

#Final Validation: Confirmed table schema preserves native float data type for revenue
metrics and successfully converted sales_date (STRING) to (Date)
```

ANALYSIS PHASE

Data Analysis Process & Metric Derivation

To establish a reliable baseline for supply chain optimization, raw transaction and inventory records cannot be compared directly due to daily fluctuations. The analytical process requires establishing historical mathematical averages. This standardizes time horizons and demand velocities across all distinct product lines, consolidating the source files into a unified intermediate table structure consisting of six key operational indicators:

Step 1 - Intermediate new table (product_order_insights) baseline Schema

- product_id
- product_name
- unit_cost
- unit_price
- avg_units_sold_per_day
- average_remaining_shelf_life
- average_inventory_in_stock
- Average_supplier_lead_time_days
- avg_days_in_stock

Calculation: avg_units_sold_per_day

```
SELECT
product_id,
CAST(AVG(quantity_sold) AS INT64) as avg_units_sold_per_day
FROM `skilful-answer-498915-k5.pharx_data.cleaned_fact_sales`
GROUP BY product_id
```

Calculation: average_remaining_shelf_life

```
SELECT
product_id,
CAST(AVG(DATE_DIFF(expiry_date, log_date, DAY)) AS INT64) as
average_remaining_shelf_life
FROM `skilful-answer-498915-k5.pharx_data.cleaned_fact_inventory_logs`
GROUP BY product_id
```

Calculation: **average_inventory_in_stock**

```
SELECT
product_id,
CAST(AVG(in_stock) AS INT64) AS average_inventory_in_stock
FROM `skilful-answer-498915-k5.pharx_data.cleaned_fact_inventory_logs`
GROUP BY product_id
```

Calculation: **Average_supplier_lead_time_days**

```
SELECT
product_id,
CAST(AVG(supplier_lead_time_days) AS INT64) AS average_supplier_lead_time_days
FROM `skilful-answer-498915-k5.pharx_data.cleaned_fact_inventory_logs`
GROUP BY product_id
```

To calculate **Avg_days_in_stock**, we divide the average inventory by the average daily sales speed. Because these metrics live in two completely separate summarized views, we must combine them using an '**INNER JOIN**' on the product_id field.

Calculation: **Avg_days_in_stock** (*average_inventory_in_stock / avg_units_sold_per_day*)

```
SELECT
i.product_id,
i.average_inventory_in_stock,
s.avg_units_sold_per_day,

#Calculation:
CAST(i.average_inventory_in_stock / s.avg_units_sold_per_day AS INT64) AS
avg_days_in_stock

FROM `skilful-answer-498915-k5.pharx_data.avg_fact_inventory_logs` AS i
INNER JOIN `skilful-answer-498915-k5.pharx_data.avg_units_sold_per_day` AS s
ON i.product_id = s.product_id;
```

Step 2: Order Frequency & Volume Modeling

With the baseline averages established in the new table, the next phase calculates the most efficient order quantities and frequencies required to eliminate overstocking and stockouts. Two new operational metrics were created:

- `place_order_every_x_days`
- `order_quantity`

'place_order_every_x_days'

Purpose: Establishes an automated ordering frequency window to prevent capital stagnation and premature restocks.

Calculation: $(\text{average_inventory_in_stock} / \text{avg_units_sold_per_day}) - \text{average_supplier_lead_time_days} - 1$

```
CAST(((i.average_inventory_in_stock / s.avg_units_sold_per_day) -  
i.average_supplier_lead_time_days - 1 AS INT64) AS place_order_every_x_days,
```

'order_quantity'

Purpose: Defines the precise volume boundary needed to maintain optimum shelf levels without introducing safety stock deficits.

Calculation: $(\text{place_order_every_x_days} * \text{avg_units_sold_per_day}) +$
 $(\text{average_supplier_lead_time_days} * \text{avg_units_sold_per_day})$

```
CAST((((i.average_inventory_in_stock / s.avg_units_sold_per_day) -  
i.average_supplier_lead_time_days - 1) * s.avg_units_sold_per_day) +  
(i.average_supplier_lead_time_days * s.avg_units_sold_per_day) AS INT64) AS  
order_quantity,
```

Table 'product_order_insights' schema after adding order frequency and volume metrics:

- product_id
- product_name
- unit_cost
- unit_price
- avg_units_sold_per_day
- average_remaining_shelf_life
- average_inventory_in_stock
- Average_supplier_lead_time_days
- Avg_days_in_stock
- Place_order_every_x_days ←
- Order_quantity ←

Step 3: Current vs. Optimised expenditure

Now that the optimal order frequency and volume were established for each product, the next phase was to calculate the exact financial impact this optimization would have for PHARx compared to their historical inventory management. Two additional metrics:

- Current_inventory_value
- Optimized_order_value

were included in the dataset. This explicitly reveals the total amount currently spent on a product versus how much PHARx should spend under the optimized replenishment parameters.

Current_inventory_value

Purpose: Calculates the total amount spent on purchasing each product

Calculation: `average_inventory_in_stock * unit_cost`

```
ROUND(i.average_inventory_in_stock * p.unit_cost, 2) AS current_inventory_value
```

Optimized_order_value

Purpose: Calculates the total amount that would have been spent under the optimized parameters.

Calculation: `order_quantity * unit_cost`

```
ROUND(order_quantity * unit_cost, 2) AS optimized_order_value
```

Table 'product_order_insights' schema after adding current vs optimised order value metrics:

- product_id
- product_name
- unit_cost
- unit_price
- avg_units_sold_per_day
- average_remaining_shelf_life
- average_inventory_in_stock
- Average_supplier_lead_time_days
- Avg_days_in_stock
- Place_order_every_x_days
- Order_quantity
- Current_inventory_value ←
- Optimized_order_value ←

Step 4: Optimisation impact

By summing the total value spent on inventory historically and comparing it with the total value that would have been spent by applying this optimization model, the analysis reveals a clear cost reduction:

- **Total** current inventory value: £6,240,881.18
- **Total** optimised order/ inventory value: £5,868,245.05
- **Net Capital saved:** £372,636.13 (6% efficiency improvement)

Core Takeaway

Implementing the optimized order frequencies and volumes removes excess inventory without creating stock deficits. For PHARx, this structural adjustment yields an immediate reduction in product expenditure by approximately 6% based purely on historical 2025 operational data.

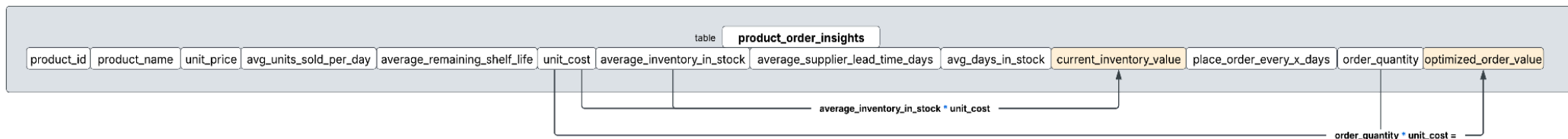
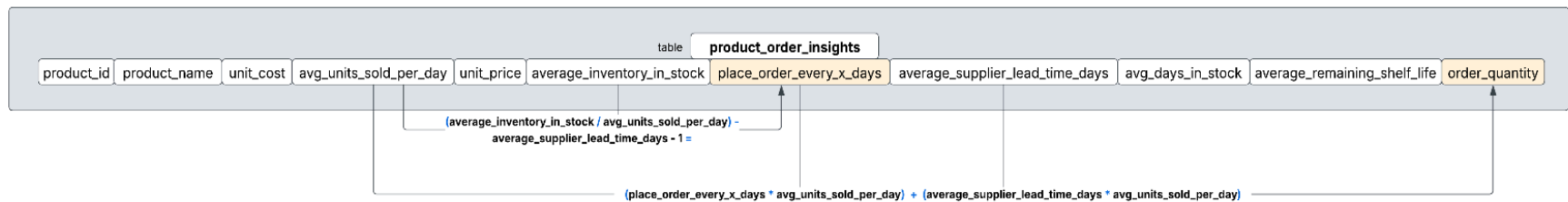
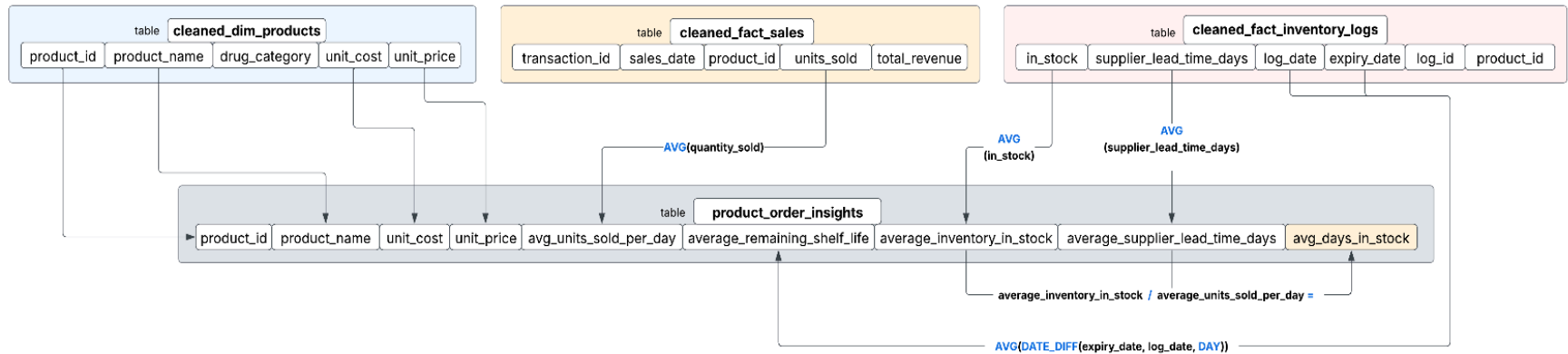
Moving forward, this model can be run continuously to adapt to fluid shifts in supply and demand. Because market conditions vary from year to year, static historical ordering patterns quickly become ineffective. Automating this process ensures procurement remains perpetually aligned with real-time requirements.

Master Query Compilation

This query combines all previous calculations into a single query to generate the final product_order_insights table. It unifies all metrics into one comprehensive dataset, serving as the exact table from which the final savings insights were generated.

```
SELECT
    i.product_id,
    p.product_name,
    p.unit_cost,
    p.unit_price,
    s.avg_units_sold_per_day as average_units_sold_per_day,
    i.average_remaining_shelf_life,
    i.average_inventory_in_stock,
    i.average_supplier_lead_time_days,
    CAST(i.average_inventory_in_stock / s.avg_units_sold_per_day AS INT64) AS
avg_days_in_stock,
    -- Advanced Metric 1: Place Order Every (X) Days
    CAST(((i.average_inventory_in_stock / s.avg_units_sold_per_day) -
i.average_supplier_lead_time_days - 1 AS INT64) AS place_order_every_x_days,
    -- Advanced Metric 2: Order Quantity
    CAST((((i.average_inventory_in_stock / s.avg_units_sold_per_day) -
i.average_supplier_lead_time_days - 1) * s.avg_units_sold_per_day) +
(i.average_supplier_lead_time_days * s.avg_units_sold_per_day) AS INT64) AS
order_quantity,
    -- Financial Comparison 1: Current Capital Tied Up (Before)
    ROUND(i.average_inventory_in_stock * p.unit_cost, 2) AS current_inventory_value,
    -- Financial Comparison 2: Optimized Capital Required (After)
    ROUND(
        CAST((((i.average_inventory_in_stock / s.avg_units_sold_per_day) -
i.average_supplier_lead_time_days - 1) * s.avg_units_sold_per_day) +
(i.average_supplier_lead_time_days * s.avg_units_sold_per_day) AS INT64)
        * p.unit_cost, 2
    ) AS optimized_order_value
FROM `skilful-answer-498915-k5.pharx_data.avg_fact_inventory_logs` AS i
INNER JOIN `skilful-answer-498915-k5.pharx_data.avg_units_sold_per_day` AS s
    ON i.product_id = s.product_id
INNER JOIN `skilful-answer-498915-k5.pharx_data.cleaned_dim_products` AS p
    ON i.product_id = p.product_id;
```

Final Schema Architecture



Total current inventory value = product_id * current_inventory_value = \$6,240,881.18

Total optimised order/ inventory value = product_id * optimized_order_value = \$5,868,245.05

Net Capital saved: \$372,636.13 = (6% efficiency improvement)

SHARE PHASE

1. Visualisation & Communication Strategy

To effectively bridge the gap between complex SQL database logic and executive decision-making, the compiled tracking metrics from the `product_order_insights` table were translated into slides:

The Executive Presentation: An 11-slide, high-contrast minimalist deck tailored specifically for the Head of Logistics and Corporate Finance. It focuses strictly on bottom-line financial impact, category-level risk, and strategic execution timelines, keeping raw code abstracted.

2. Core Visual Insights

The data stories surfaced by the visual analysis isolated clear operational anomalies across the depot's inventory portfolio:

The Specialty Capital Drag: Visualizing the financial holding footprints exposed that slow-moving specialty medications (Oncology and Immunology lines) were responsible for 99.3% of the warehouse's frozen capital due to irregular, manual bulk-ordering patterns.

High-Volume Buffer Preservation: Conversely, high-velocity categories like Cardiology and Diabetology showed minimal holding value inflation, confirming that their primary operational threat was stockout instability caused by rigid ordering intervals rather than overstocking.

ACT PHASE

1. Final Conclusions & High-Level Recommendations

The 12-month operational audit proves that static, historical purchasing patterns are structurally incompatible with fluid pharmaceutical supply chains. To permanently secure working capital and eliminate stockout risks, the client should execute three strategic actions:

Embed Continuous Execution: Integrate the BigQuery replenishment query architecture directly into daily standard operating procedures (SOPs) so order cycles dynamically adapt as sales velocities fluctuate.

Reallocate Released Liquidity: Reinvest the unlocked **£372,636.13** in standing inventory savings directly into procurement automation software and warehouse scaling initiatives without drawing on external financing.

Transform to Demand-Driven Auto-Procurement: Build the SQL model out into an auto-ordering tool. The system will automatically generate purchase requests based on live product demand, calculating active product expiry profiles, vendor lead times, and emergency buffers prior to execution.

2. Automated Expansion Signaling

By configuring the automated procurement tool to cross-reference optimal replenishment volumes with live warehouse capacity limits, the software serves as an objective, data-backed operational trigger. If the engine flags that optimal stock volumes cannot be physically accepted due to a lack of shelf space, it provides corporate leadership with empirical justification that physical depot expansion is economically viable—self-funded by the savings achieved from this initial optimization.

3. Future Deliverables for Further Exploration

To build upon these findings and expand operational visibility, the following future deliverables are recommended for deployment:

- **Live Executive Tracking Dashboard:** A real-time dashboard connected directly to BigQuery to monitor active category shelf-life decay rates and automatically flag SKUs nearing critical expiration windows.
- **Qualitative Vendor Audit:** A structured scorecard system to evaluate upstream supplier reliability, mapping actual delivery variances against stated lead times to continuously refine safety stock cushions.